

Implement Client Script & UI Policy (Incident)

Milestone 1: Create UI Policy on Incident

Description

Create a UI Policy on the Incident table to dynamically control the behavior of form fields based on specific conditions. Configure the UI Policy to make selected fields mandatory, read-only, or visible when the defined condition is met. This helps ensure that users provide the required information and follow consistent data-entry practices while creating or updating Incident records.

Activity 1: Create UI Policy

Description

1. Log in to the **ServiceNow instance** with administrative or configuration access.
2. Navigate to:
System UI → UI Policies
3. Click **New** to create a new UI Policy record.
4. Fill in the following details:
 - **Name:** High Impact Control
This name clearly represents the purpose of the policy, which is to apply additional controls when an Incident has a high impact.
 - **Table:** Incident
Specifies that the UI Policy applies to Incident records.
 - **Active:** true
Enables the UI Policy so it is enforced on the Incident form.

5. Configure the **Conditions** section as follows:

- **Field:** Impact
- **Operator:** is
- **Value:** 1 – High

6. This condition ensures the UI Policy is triggered only when the Incident impact is set to High.

7. And Ui policy action Assignment group

8. Check mandatory

9. Set **Reverse if false** to **true**.

This ensures that when the condition is not met, any field changes applied by the policy are automatically reverted.

10. Review the configuration to ensure accuracy.

11. Click **Submit** to save the UI Policy.

Short description	Table	Conditions	Reverse if false	On load	Updated	Order
(empty)	Email Filter [sys_email_filter]		true	true	2026-06-13 02:36:26	100
(empty)	Service commitment [service_commitment]	type=maintenance_window^EQ	true	true	2010-04-30 10:37:35	100
(empty)	Dictionary Entry [sys_dictionary]	dynamic_creation=true^EQ	true	true	2008-10-03 13:02:45	100
(empty)	Service commitment [service_commitment]	type=availability^EQ	true	true	2010-02-18 17:26:05	100
Max entries only available for multi entry parameters	UI parameter [sys_sg_ui_parameter]	multi_entry=false^EQ	true	true	2019-07-16 10:57:04	110
Set value for reference and make it readonly for requested_for type			true	true	2024-09-16 00:52:17	150
UI policy to hide the dependent route table	UX App linked routes [sys_ux_dependent_route]		true	true	2025-03-11 02:58:21	100
"Script type" field is visible only when "Supports stand request" is true	PAR Metadata [par_metadata]	supports_standard_request=true	true	true	2024-08-27 05:39:02	100

servicenow

AllFavoritesHistoryAdmin

UI Policy - High Impact Control

Search

UpdateDelete

UI PolicyHigh Impact Control

UpdateDelete

UI policies change fields on a form based on a set of conditions. Use UI policies to show or hide fields, or to make fields mandatory or read-only based on these conditions. A UI policy specifies one or more actions to take when the policy is triggered. First, create the policy. Then add as many actions as necessary. UI policy actions are applied only if all of the following conditions are met:

- The UI Policy is **Active**
- The items in the **Conditions** field evaluate to true
- The field specified in the UI policy action is present on the specified form

[More info](#)

TableIncident [incident]

ApplicationGlobal

Active☒

* Short description

High Impact Control

When to Apply

Conditions

Add Filter ConditionAdd OR Clause

Impactis1 - HighANDOR

UpdateDelete

Related Links

servicenow

AllFavoritesHistoryAdmin

UI Policy Action - New Record

Search

Submit

UI Policy ActionNew record

Submit

UI policy actions specify exactly what actions to take on a specified field. The conditions specified in the UI policy determine when these actions are triggered. [More Info](#)

UI policyHigh Impact Control

ApplicationGlobal

TableIncident [incident]

MandatoryTrue

* Field nameAssignment group

VisibleLeave alone

Read onlyFalse

Clear the field value☐

Submit

Implement Client Script & UI Policy (Incident)

Milestone 2: Create UI Policy Action – Urgency

Description

Create a **UI Policy Action** for the **Urgency** field on the Incident form to control its behavior based on the conditions defined in the UI Policy. Configure the action to make the Urgency field **mandatory, visible, or read-only** as required. This ensures that the appropriate urgency information is captured consistently and helps users prioritize incidents accurately.

Activity 1: Configure the Urgency Field

Description

1. Open the UI Policy Actions

Go to the **High Impact Control** UI Policy you created earlier. Scroll down to the **UI Policy Actions** related list. This is where you define the specific behavior of fields when the policy conditions are met.

2. Create a New Action

Click **New** to create a UI Policy Action. This lets you configure how individual fields behave when the UI Policy is triggered.

3. Fill in the Action Details

- **Field name:** Urgency
Select the **Urgency** field on the Incident form. This is the field you want to control.
- **Read-only:** true
Check this option to make the Urgency field read-only whenever the UI Policy condition is met (Impact = High). Users will be able to see the field but cannot change its value.
- **Visible:** Leave this field as is (do not change).
By leaving it unchanged, the field will remain visible on the form, but its behavior will be controlled by the Read-only setting.

4. Submit the Action

Click **Submit** to save the UI Policy Action. The action is now linked to the High Impact Control UI Policy and will automatically apply to the Urgency field when an Incident has High Impact.

5. Optional Verification

To test the action:

- Open an Incident record.
- Set the **Impact** field to **High**.
- Check that the **Urgency** field becomes read-only.
- Change Impact to a different value to confirm that the field reverts back if needed.

The screenshot shows the 'UI Policy Action - New Record' form in ServiceNow. The form is titled 'UI Policy Action - New Record' and includes a search bar and navigation links. The form fields are as follows:

- UI policy:** High Impact Control
- Table:** Incident [Incident]
- * Field name:** Urgency
- Application:** Global
- Mandatory:** Leave alone
- Visible:** Leave alone
- Read only:** True
- Clear the field value:** ☐

A blue banner at the top of the form states: 'UI policy actions specify exactly what actions to take on a specified field. The conditions specified in the UI policy determine when these actions are triggered. [More Info](#)'.

The form includes a 'Submit' button at the bottom left and a 'Submit' button at the top right.

Implement Client Script & UI Policy (Incident)

Milestone 3: Create onChange Client Script

Description

Create an **onChange Client Script** on the Incident form to dynamically perform an action whenever a specified field value is changed. The script should use the **g_form API** to update related fields or display appropriate messages based on the selected value. This helps automate form behavior, improve data accuracy, and provide users with a more interactive Incident form experience.

Activity 1: Create Client Script

Description

1. **Navigate to Client Scripts:**

Go to System UI → Client Scripts.

2. **Click New** to create a new Client Script.

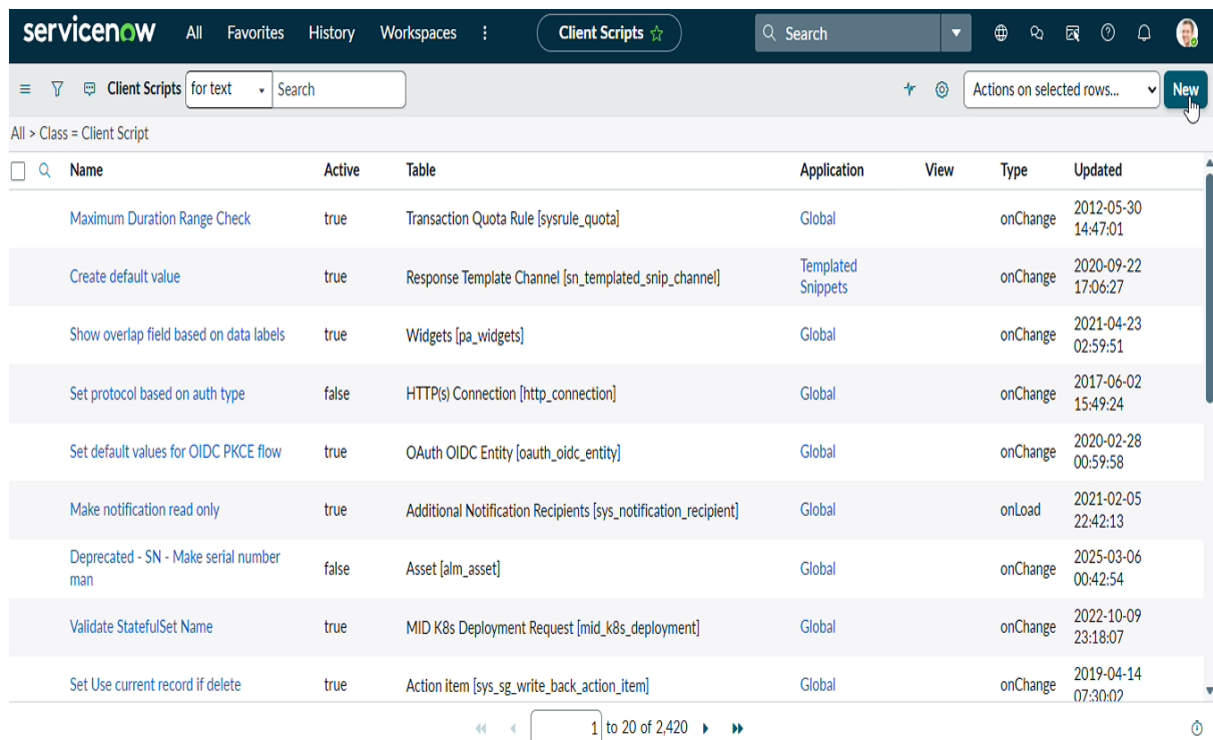
3. **Fill in the details:**

- **Name:** Auto set urgency for high impact – clearly shows the purpose.
- **Table:** Incident – the script runs only on Incident records.
- **Type:** onChange – executes when a field value changes.
- **Field name:** Impact – triggers the script when Impact is updated.
- **Active:** true – ensures the script runs immediately.

4. **Paste the script** in the script editor section.

```
function onChange(control, oldValue, newValue, isLoading) {  
  if (isLoading || newValue === '') {  
    return;  
  }  
  if (newValue === '1') {  
    g_form.setValue('urgency', '1');  
    g_form.addInfoMessage('Urgency set to High for High impact incident.');
```

5. **Click Submit** to save the Client Script.
6. **Verify:** Open an Incident record and change the Impact field to High. The Urgency field should update automatically
7. **Click Submit.**



The screenshot shows the ServiceNow interface for Client Scripts. The top navigation bar includes 'servicenow', 'All', 'Favorites', 'History', 'Workspaces', and a 'Client Scripts' button. A search bar is present. Below the navigation bar, there's a filter section with 'Client Scripts' and a search input. The main area displays a table of Client Scripts. The table has columns: Name, Active, Table, Application, View, Type, and Updated. The table lists several scripts, including 'Maximum Duration Range Check', 'Create default value', 'Show overlap field based on data labels', 'Set protocol based on auth type', 'Set default values for OIDC PKCE flow', 'Make notification read only', 'Deprecated - SN - Make serial number man', 'Validate StatefulSet Name', and 'Set Use current record if delete'. The 'Updated' column shows dates and times. At the bottom, there's a pagination bar showing '1 to 20 of 2,420'.

Name	Active	Table	Application	View	Type	Updated
Maximum Duration Range Check	true	Transaction Quota Rule [sysrule_quota]	Global		onChange	2012-05-30 14:47:01
Create default value	true	Response Template Channel [sn_templated_snip_channel]	Templated Snippets		onChange	2020-09-22 17:06:27
Show overlap field based on data labels	true	Widgets [pa_widgets]	Global		onChange	2021-04-23 02:59:51
Set protocol based on auth type	false	HTTP(s) Connection [http_connection]	Global		onChange	2017-06-02 15:49:24
Set default values for OIDC PKCE flow	true	OAuth OIDC Entity [oauth_oidc_entity]	Global		onChange	2020-02-28 00:59:58
Make notification read only	true	Additional Notification Recipients [sys_notification_recipient]	Global		onLoad	2021-02-05 22:42:13
Deprecated - SN - Make serial number man	false	Asset [alm_asset]	Global		onChange	2025-03-06 00:42:54
Validate StatefulSet Name	true	MID K8s Deployment Request [mid_k8s_deployment]	Global		onChange	2022-10-09 23:18:07
Set Use current record if delete	true	Action item [sys_sg_write_back_action_item]	Global		onChange	2019-04-14 07:30:02

servicenow

AllFavoritesHistory

Client Script - Auto set urgency for h...

Search

Client Script

Auto set urgency for high impact

UpdateDelete

Additional actions

New client scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.

Name

Auto set urgency for high impact

Table

Incident [incident]

UI Type

Desktop

Type

onChange

Field name

-- None --

Application

Global

Active

☒

Inherited

☐

Global

☒

Description

Messages

Script

1function onChange(control, oldValue, newValue, isLoading) {

2 if (isLoading || newValue == '') {

3 return;

4 }

5

6 if (newValue == '1') {

Implement Client Script & UI Policy (Incident)

Milestone 4: Create onSubmit Client Script (Save Validation)

Description

Create an **onSubmit Client Script** on the Incident form to validate the information entered by the user before the record is saved or submitted. Use the **g_form API** to check whether the required conditions are satisfied and prevent submission when invalid or incomplete data is found. Display an appropriate message to guide the user in correcting the information before saving the Incident.

Activity 1: Create Client Script

Description

1. **Navigate to Client Scripts:**

Go to System UI → Client Scripts in your ServiceNow instance.

2. **Click New** to create a new Client Script.

3. **Fill in the details:**

- **Name:** Prevent save if Assigned To missing – clearly indicates that the script prevents saving if the field is empty.
- **Table:** Incident – applies the script only to Incident records.
- **Type:** onSubmit – executes when the user tries to save the record.
- **Active:** true – ensures the script is enforced immediately.

4. **Paste the script** into the script editor section. This script should check if the **Assigned To** field is empty and prevent the record from saving if it is.

```
function onSubmit() {  
    if (g_form.getValue('impact') == '1' &&  
        g_form.getValue('assigned_to') == '') {  
        g_form.showErrorMessage(  
            'assigned_to',  
            'Assigned To is mandatory for High impact incidents.'  
        );  
        return false;  
    }  
    return true;  
}
```

5. **Click Submit** to save the Client Script.
6. **Verify:** Try saving an Incident without filling the **Assigned To** field. The script should prevent the save and display a warning.
7. **Click Submit.**

The screenshot shows the ServiceNow 'Client Script - New Record' configuration page. The title bar indicates the script is for 'Prevent save if Assigned To missing'. The configuration fields on the left include: Name (Prevent save if Assigned To missing), Table (Incident [incident]), UI Type (Desktop), Type (onSubmit), and Field name (empty). The right-hand configuration area shows: Application (Global), Active (checked), Inherited (unchecked), Global (checked), and View (empty). The Script editor at the bottom contains the following JavaScript code:

```
function onSubmit() {  
    if (g_form.getValue('impact') == '1' &&  
        g_form.getValue('assigned_to') == '') {  
        g_form.showErrorMessage(  
            'assigned_to',  
            'Assigned To is mandatory for High impact incidents.'  
        );  
        return false;  
    }  
    return true;  
}
```

Implement Client Script & UI Policy (Incident)

Milestone 5: Create onCellEdit Client Script

Description

Create an **onCellEdit Client Script** on the Incident table to control and validate field values when users edit Incident records directly from a **list view**. The script should check the updated value and allow or prevent the change based on the defined conditions. This helps maintain data accuracy and ensures that users follow the required validation rules even when making inline edits from the Incident list.

Activity 1: Create Client Script

Description

1. Navigate to Client Scripts:

Go to System UI → Client Scripts in your ServiceNow instance.

2. Click New to create a new Client Script.

3. Fill in the details:

- **Name:** Prevent state change via list edit – clearly indicates that the script prevents users from changing the State field directly in a list.
- **Table:** Incident – applies the script only to Incident records.
- **Type:** onCellEdit – runs whenever a field is edited directly in a list view.
- **Field name:** State – the script will monitor changes to the State field.
- **Active:** true – ensures the script is enforced immediately.

4. **Paste the script** into the script editor section. The script should prevent unauthorized changes to the State field when editing from the list view.

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback)

{

    alert ('State cannot be updated using list editing. Please open the Incident.');
```

```
    callback(false);

}
```

5. **Click Submit** to save the Client Script.
6. **Verify:** Open an Incident list and try to change the State field directly in the list. The script should block the change as intended.
7. **Click Submit.**

The screenshot shows the ServiceNow 'Client Script - New Record' form. The form is titled 'Client Script - Prevent state change via list edit'. It includes fields for Name, Table, UI Type, Type, Field name, Application, Active, Inherited, Global, and View. The Name field is 'Prevent state change via list edit', Table is 'Incident [incident]', UI Type is 'Desktop', Type is 'onCellEdit', and Application is 'Global'. The Active checkbox is checked. The Description and Messages fields are empty. The Script field contains the following code:

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    alert('State cannot be updated using list editing. Please open the Incident.');
```

```
    callback(false);
}
```

Implement Client Script & UI Policy (Incident)

Milestone 6: Testing the Configuration

Description

Test all the configured **UI Policies, UI Policy Actions, and Client Scripts** on the Incident form and list view to ensure they work as expected. Verify that fields become **mandatory, visible, read-only, or updated** based on the defined conditions. Test different scenarios, including valid and invalid inputs, form submission, field changes, and inline list edits, and confirm that the expected validation messages and behaviors are triggered correctly.

Activity 1: Test Mandatory Enforcement

Description

1. Navigate to Incident:

Go to Incident → Create New to open a new Incident form.

2. Set the Impact Field:

Select **High** for the **Impact** field. This will trigger the UI Policy and Client Script behaviors you configured earlier.

3. Leave Assigned To Empty:

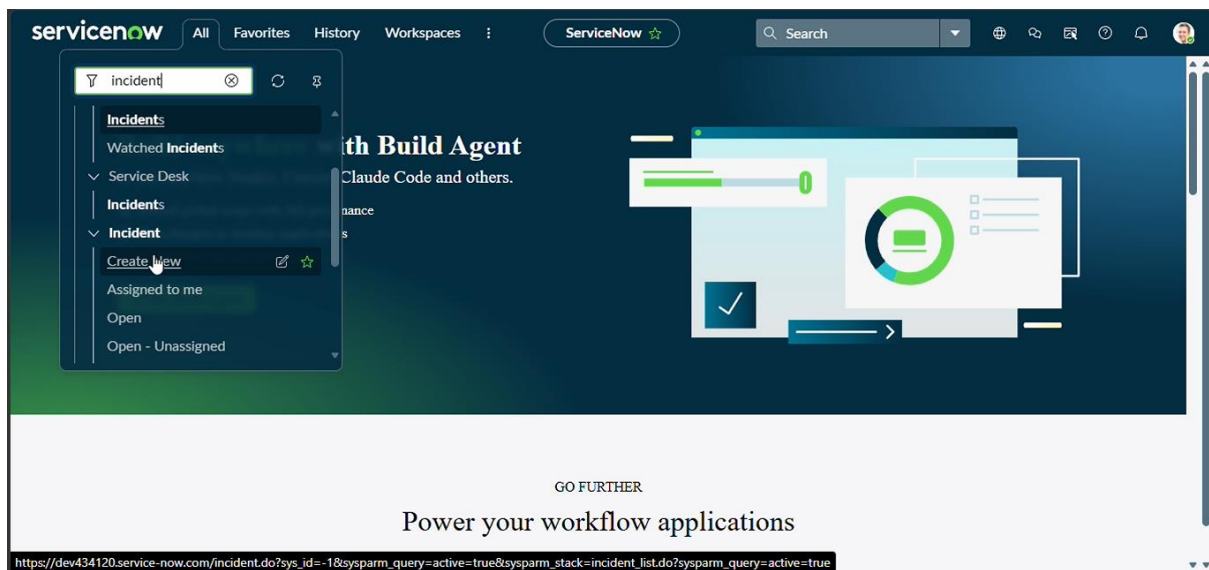
Do not fill in the **Assigned To** field. This will test the onSubmit Client Script that prevents saving if Assigned To is missing.

4. Click Submit:

Attempt to save the Incident record.

5. Verify the Result:

- The Incident should **not be saved** because Assigned To is required when Impact is High.
- An **error message** should appear, indicating that the Assigned To field cannot be empty.
- Confirm that all UI Policy actions, such as read-only fields or automated urgency updates, are working as expected.



ServiceNow Incident - Create INC0010017

The following mandatory fields are not filled in: Assignment group

Number	INC0010017	Channel	-- None --
* Caller	Adela Cervantsz	State	New
Category	Inquiry / Help	Impact	1 - High
Subcategory	-- None --	Urgency	3 - Low
Service		Priority	3 - Moderate
Service offering		* Assignment group	
Configuration item		Assigned to	
* Short description	Testing UI Policy		
Description			

Related Search Results

Related Search: Testing UI Policy

Knowledge & Catalog (All)

Activity 2: Test Successful Save

Description

1. Fill the Assigned To Field:

Open the Incident form and select a user in the **Assigned To** field. This ensures the onSubmit Client Script condition is satisfied.

2. Click Submit:

Attempt to save the Incident record.

3. Verify the Result:

- The Incident should be **saved successfully** without any errors.
- Confirm that all other configured behaviors, such as:
- Urgency auto-setting (from the onChange Client Script)
- **Read-only fields** (from the UI Policy Action)
- **State restrictions** (from the onCellEdit Client Script)
are functioning as expected

servicenow All Favorites History Admin Incident - Create INC0010017 Search

Incident New record Submit Resolve

The following mandatory fields are not filled in: Assignment group

Number INC0010017 Channel -- None --

* Caller Adela Cervantsz State New

Category Inquiry / Help Impact 1 - High

Subcategory -- None -- Urgency 3 - Low

Service Priority 3 - Moderate

Service offering * Assignment group Software

Configuration item Assigned to Beth Anglin

* Short description Testing UI Policy

Description

Related Search Results

Related Search Q Testing UI Policy Knowledge & Catalog (All)

Activity 3: Reverse Condition Test

Description

1. Open an Existing Incident:

Go to Incident → Open and select an Incident where **Impact** is currently set to **High**.

2. Change Impact:

Update the **Impact** field from **High** to **Medium**. This will test the **Reverse if false** functionality of your UI Policy and the behavior of your Client Scripts.

3. Verify the Field Behavior:

- **Assigned To** should no longer be mandatory.
- **Urgency** should now be editable, meaning the read-only UI Policy action is reversed.

4. Save the Record:

Click **Submit** to save the changes. The record should save successfully without errors, confirming that the reverse condition works as intended.

servicenow All Favorites History Admin Incident - Create INC0010012 Search

Incident New record Submit Resolve

Number INC0010012 Channel -- None --

* Caller Adela Cervantsz State New

Category Inquiry / Help Impact 2 - Medium

Subcategory -- None -- Urgency 1 - High

Service Priority 2 - High

Service offering * Assignment group

Configuration item Assigned to

* Short description testing UI policy

Description

Related Search Results

Related Search Knowledge & Catalog (All)

Activity 4: Test List Edit Blocking

Description

1. Navigate to the Incident List:

Go to Incident → All to view all Incident records in a list view.

2. Attempt a List Edit:

- Locate any Incident in the list.
- **Double-click the State field** to try editing it directly from the list.

3. Verify the Behavior:

- An **alert message** should appear, indicating that direct changes to the State field are not allowed.
- The **State value should remain unchanged**, confirming that the onCellEdit Client Script is working correctly.

</

Activity 5: Test Form-Based Update

Description

1. Open the Incident Record:

Navigate to the same Incident record you used in previous tests.

2. Change the State:

On the **Incident form**, update the **State** field to a new value. This tests that changes via the form (not the list) are allowed.

3. Click Update:

Save the record by clicking **Update**.

4. Verify the Result:

- The **State** change should be saved successfully.
- Confirm that other fields, such as **Assigned To** and **Urgency**, still follow the configured UI Policy and Client Script rules.

The screenshot shows the ServiceNow Incident form for INC0010017. The form fields are as follows:

Field	Value
Number	INC0010017
* Caller	Adela Cervantsz
Category	Inquiry / Help
Subcategory	-- None --
Service	
Service offering	
Configuration item	
* Short description	Testing UI Policy
Description	

The context menu is open, showing the following options:

- Save
- Add
- Save record and remain here
- Copy Incident
- Create Incident Task
- Create Outage
- Create Problem
- Create Request
- Create Child Incident
- Create Normal Change
- Create Standard Change
- Create Emergency Change
- Refresh Impacted Services
- Metrics Timeline
- Follow on Live Feed
- Analyze Access
- Configure
- Export
- View
- Create Favorite
- Copy URL
- Copy sys_id
- Show XML
- History

The screenshot shows the ServiceNow Incidents list view. The table contains the following data:

Number	Opened	Short description	Caller	Priority	State	Category	Assignment group	Assigned to	Updated	Updated by
INC0010017	2026-09-28 06:08:12	Testing UI Policy	Adela Cervantsz	3 - Moderate	In Progress	Inquiry / Help	Software	Beth Anglin	2026-09-28 06:26:57	admin
INC0010015	2026-09-28 00:19:19	Test Incident for Client	Abel Tuter	5 - Planning	In Progress	Inquiry / Help	(empty)	Beth Anglin	2026-09-28 00:25:03	admin
INC0010013	2026-09-27 23:58:10	Hii	Abel Tuter	3 - Moderate	In Progress	Inquiry / Help	CAB Approval	Bernard Laboy	2026-09-28 00:02:39	admin
INC0010012	2026-09-27 23:33:56	practise	Abel Tuter	5 - Planning	In Progress	Inquiry / Help	(empty)	(empty)	2026-09-28 06:36:41	admin
INC0009009	2018-08-30 01:06:16	Unable to access the shared folder.	David Miller	4 - Low	In Progress	Inquiry / Help	(empty)	(empty)	2026-09-28 00:43:35	admin
INC0009005	2018-08-31 21:35:21	Email server is down.	David Miller	1 - Critical	New	Software	(empty)	(empty)	2018-12-12 23:18:55	admin
INC0009001	2018-09-11 20:56:26	Unable to post content on a Wiki page	David Miller	3 - Moderate	New	Inquiry / Help	(empty)	(empty)	2018-12-12 23:32:42	admin
INC0008112	2019-07-29 11:48:43	Assessment : ATF Assessor	survey user	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2019-07-29 11:49:28	admin

Implement Client Script & UI Policy (Incident)

Conclusion:

Description

This project successfully demonstrates the implementation of UI Policies, UI Policy Actions, and Client Scripts on the ServiceNow Incident table. The configuration enables dynamic field behavior, validation, and controlled data entry based on user actions and predefined conditions. By testing different scenarios, the Incident form and list view become more user-friendly, consistent, and reliable, helping improve overall data quality and incident management efficiency.

The Implement Client Script & UI Policy (Incident) micro project effectively demonstrates how **UI Policies** and **Client Scripts** can work together to enforce dynamic field behavior, automate updates, and prevent incorrect data submission on Incident forms. By using `onChange`, `onSubmit`, and `onCellEdit` scripts alongside UI Policy actions, the project ensures fields like **Assigned To** and **Urgency** respond appropriately to changes in **Impact**, maintaining clean and consistent data. This solution not only enforces mandatory fields and controls user interactions but also remains lightweight, efficient, and easy to implement, making it ideal for short-duration micro projects while showcasing practical best practices for improving form usability and data integrity in ServiceNow.